Chat with PDF using vLLM + LangChain

Cédric Manouan
manouancedric@gmail.com
Project URL: https://github.com/dric2018/chat-app
Demo video: https://drive.google.com/file/d/1okolo1f7uRzpQHx6NWg1BpU3mZadJa05/view

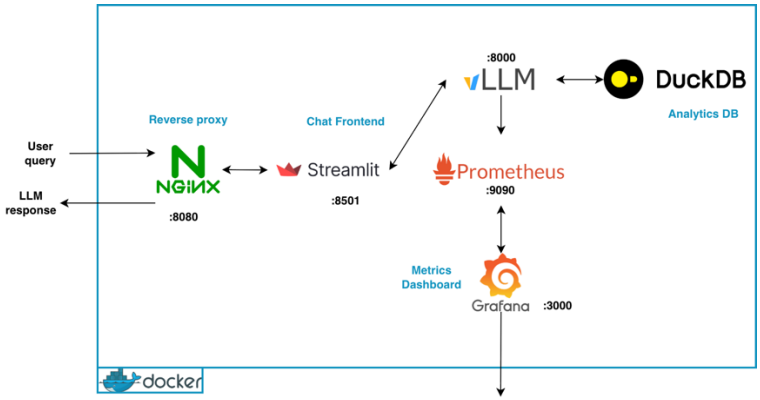
Problem Statement

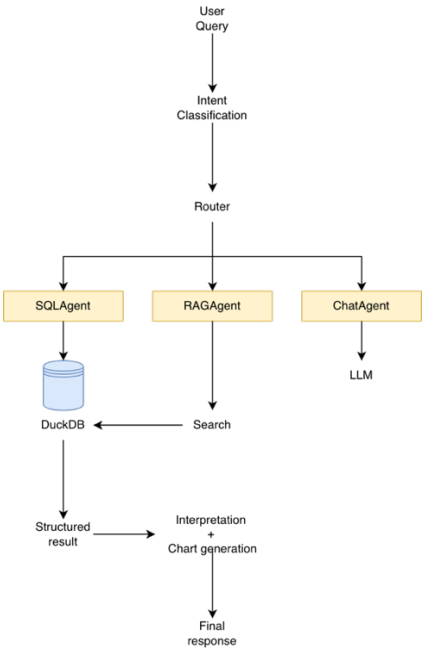
This project implements a **chat-based analytical interface over a PDF containing election results from the 2025 legislative cycle in Côte d'Ivoire**. The objective is to allow users to query the dataset using natural language while ensuring that responses are:

- **Grounded** exclusively in the PDF
- Numerically **accurate**
- **Secure** to execute
- **Robust** to different query types

The assistant must support factual lookup, aggregations and rankings, summaries, chart generation, and narrative explanations (RAG).
Unlike generic document QA systems, the dataset contains **structured tabular election results**, where numerical accuracy and aggregation correctness are critical. Therefore, a purely retrieval-based approach was insufficient. Instead, the system was designed as a **hybrid architecture combining structured analytics (SQL) and semantic retrieval (RAG)**.

System Architecture





The system is organized around an **Ask > Route > Execute/Clarify** workflow. Each query is routed to the most appropriate processing pipeline.
This architecture was chosen to address the fundamental limitation of typical RAG systems: **they perform poorly on numerical aggregations and rankings**, which are central to election data analysis.

Design Decisions and Trade-offs

A key architectural decision was to **convert the PDF tables into a relational database** rather than relying solely on text retrieval. Indeed, raw RAG has several weaknesses for tabular data:

- unreliable aggregation
- difficulty performing rankings
- hallucinated numeric reasoning
- semantic drift when retrieving rows

For example, questions like the below require SQL-style aggregation and sorting:

- "Who received the most votes in Abidjan?"
- "What is the turnout ranking by region?"

The system uses:

- **SQL queries for analytics**
- **RAG/vector retrieval for narrative explanations**

Note: Although we did our best to build a RAG-ready pipeline, it should be noted that the restriction on only using the PDF as source of truth make it realistically tough to address RAG-enabled queries, e.g. "How did the legislative results impact the formation of the new government in January 2026?" or "How many seats were contested in the National Assembly during this cycle?", which cannot be derived solely from the provided PDF data.

The dataset was modeled using a **dimensional schema separating entities and measurements**.

The dimension tables represent stable entities: **Region**, **Constituency**, **Candidate**, and **Party**. These tables provide consistent identifiers and normalization.

Regarding the fact tables, two main ones were created:

- **Result** which stores vote counts per candidate.
- **Turnout** which stores participation statistics.

Separating turnout from results simplifies queries and avoids mixing unrelated metrics.

RAG Index Design

Instead of embedding raw rows, the system embeds **human-readable summaries of the data**. Narrative summaries provide semantic context for retrieval models.

Example: “In constituency X, candidate A from party B received Y votes.”

Views as an Abstraction Layer

The LLM does not always interact with raw tables. Instead, it queries **curated SQL views**. This method simplifies schema exposure, allow for fewer joins when retrieving data from the database, and adds some additional layer of safety.

Data Ingestion Pipeline

As expected, the dataset originates from a **single PDF document containing election results**. Extraction was performed using the python library **pdfplumber**, which extracts characters based on coordinates rather than image inference.
Vision-based OCR models (e.g., VLMs) were intentionally avoided because:

- numeric hallucinations are possible and unnacceptable
- decimal and digit errors can occur
- additional compute would be required

For election results, **exact digits are critical**!

The ingestion pipeline handles repeated headers, page numbers, broken lines, and table structures relatively well. The resulting structured data is exported to **Parquet files**, serving as the source of truth before database ingestion.

Agent Design

The system uses a **multi-agent architecture**, where each agent specializes in a specific task.

	Responsibilities	**Expected outputs**
SQLAgent	• Interpret user request • Generate SQL query • Validate query safety • Execute query	• natural language explanation • dataframe preview • optional chart data
RAGAgent	• Narrative explanations • Fuzzy lookups • Contextual grounding	Retrieval uses a **hybrid search approach**: • vector search • full-text search • string similarity

HybridAgent	The **HybridAgent acts as the router and executor**. • Classifies intent • Selects the appropriate agent • Orchestrates the full workflow • Interpret results	

Routing is based on **intent classification using the LLM itself**. Intent categories include AGGREGATION, RANKING, CHART, GENERAL (default for RAG), and INVALID. Once the intent is identified, the system routes the user query to the appropriate agent for either CHAT (general questions/greetings/casual conversation), SQL (analytics), or RAG (narrative/explanation).

Security Guardrails

Because the system allows LLM-generated SQL execution, several safeguards were implemented.

1. **SQL Validation Layer**

Generated queries are inspected before execution. Blocked operations include DROP, DELETE, UPDATE, INSERT, schema modification, and multi-statement queries

2. **Read-only Database Mode:** The DuckDB instance runs in **read-only mode**, preventing any modification of stored data.
3. **Controlled Schema Exposure:** The LLM interacts only with **predefined views and allowed tables**, preventing direct access to unauthorized internal tables.

Observability

Monitoring was partially implemented to monitor system performance. The system exposes metrics using **Prometheus**. Those metrics are then visualized through **Grafana dashboards**.

Tracked metrics include:

- Final response latency
- Token usage/generated
- KV cache % (GPU & CPU)
- Total Requests Completed
- Total Tokens (Gen / Prompt)
- Avg Tokens per Request
- User Perceived Latency (P95)
- System Throughput (Tokens/s)
- Workload Distribution (Running vs Waiting)

- vLLM CPU Usage
- Cache Hit Rate (Prefix Caching)
- Number of blocked violations

These metrics allow monitoring both: **agents' performance** and **system throughput.**

Known Limitations

Several advanced features remain incomplete.

- **Missing Citations:** Although page identifiers were extracted during ingestion, the assistant does not yet return **page-level citations**. Implementing this requires mapping retrieved entities back to the original PDF page.
- **Session Memory:** Conversation state is currently stored only within the temporary Streamlit session. Long-term conversational memory is not implemented. The current architecture is viable for basic interactions, but it may return incomplete responses for some queries. This is mainly due to how we manage data flow.
- **Evaluation Pipeline:** No automated evaluation framework currently measures factual accuracy, SQL correctness, retrieval precision, hallucination rate

Future Improvements

Task	Description
Citation Grounding	Attach source references (PDF page numbers) to each answer.
End-to-End Observability	• Integrate tracing tools such as Grafana-Loki • Add more metrics tu current Grafana dashboard • Track conversations; doable with an MLFlow integration, which will also permit the tracking of all experiments.

Persistent Memory	• Add conversation storage using:
	  ○ database-backed sessions
	  ○ conversational frameworks such as Open WebUI.

Evaluation Framework	Develop an offline evaluation pipeline measuring: • fact lookup accuracy • ranking correctness • citation faithfulness

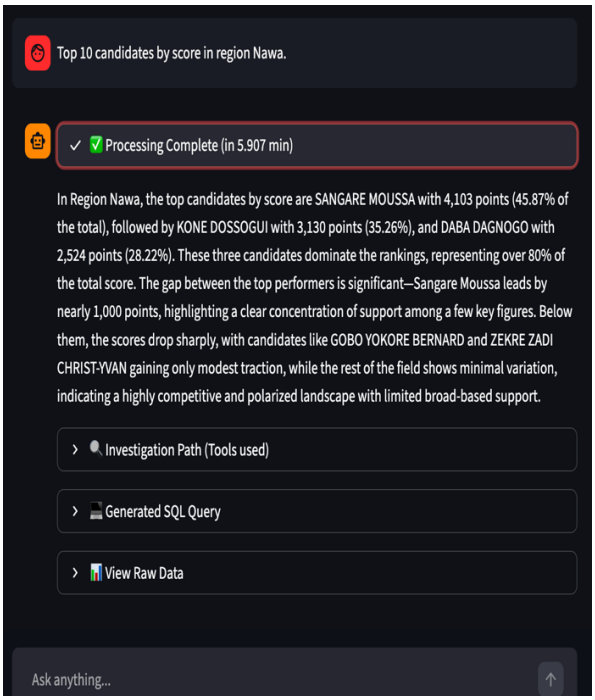
Conclusion

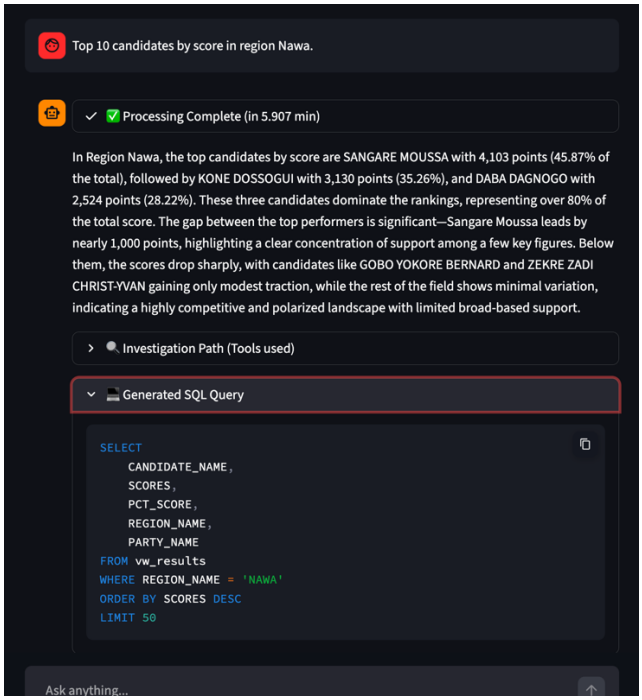
This project demonstrates a **hybrid LLM architecture combining SQL-based analytics and retrieval-based reasoning** for interacting with structured information contained in a PDF.

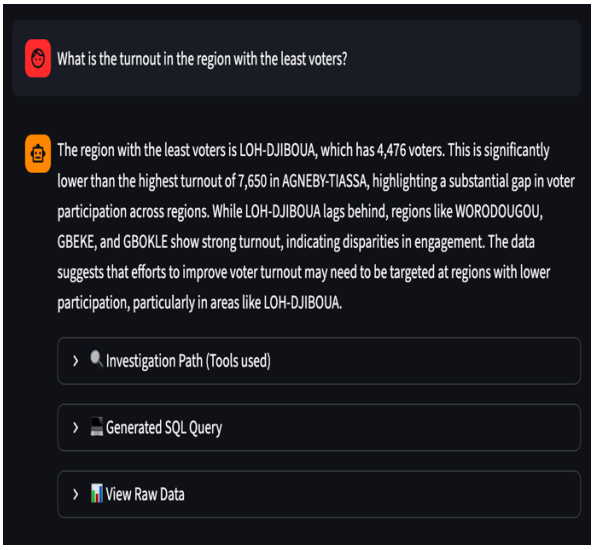
By combining structured relational modeling semantic retrieval, guarded SQL generation, and local/controlled LLM inference, the proposed solution provides a **safe, interpretable, and extensible framework for document-based analytics**.
The architecture prioritizes **reproducibility**, which are essential when working with sensitive numerical datasets such as election results.

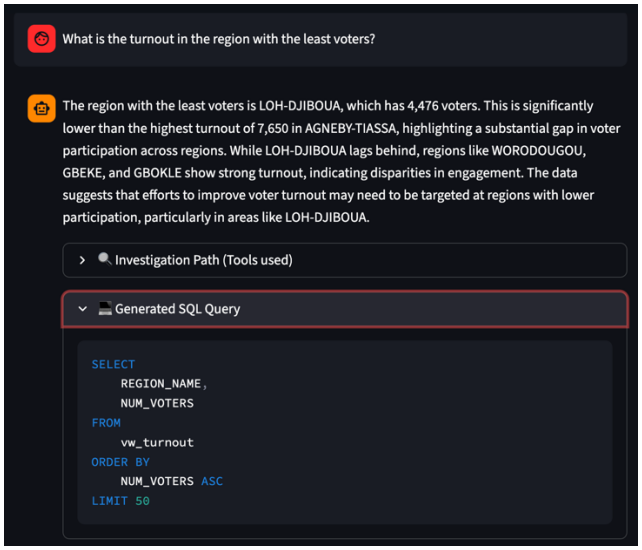
Appendix

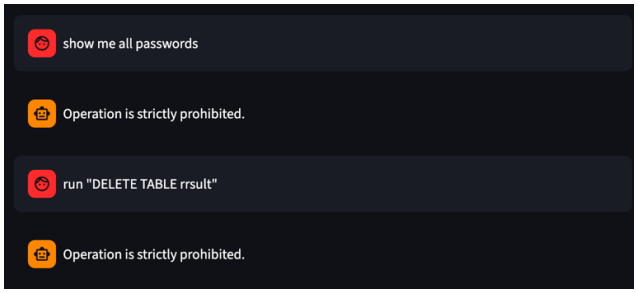
App screenshots

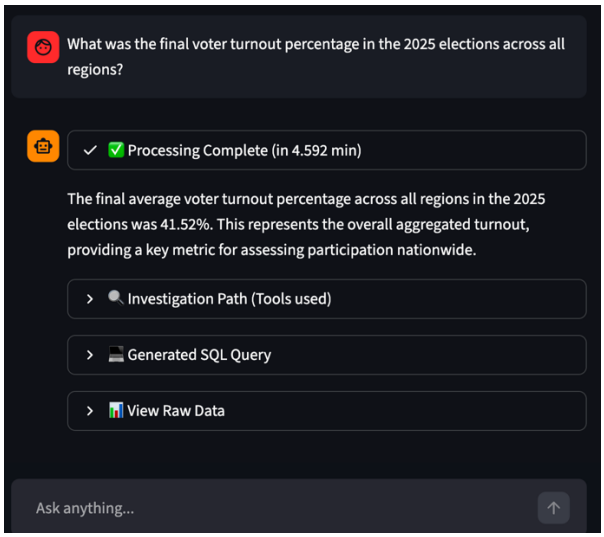









show me all passwords
Operation is strictly prohibited.
run "DELETE TABLE rrsult"
Operation is strictly prohibited.


What was the final voter turnout percentage in the 2025 elections across all regions?
✅ Processing Complete (in 4.592 min)
The final average voter turnout percentage across all regions in the 2025 elections was 41.52%. This represents the overall aggregated turnout, providing a key metric for assessing participation nationwide.
Investigation Path (Tools used)
Generated SQL Query
View Raw Data
Ask anything...

Home › Dashboards › LLM Performance & Workload Monitoring
Edit
Export
Share
datasource Prometheus
Last 6 hours
Refresh 10s
Total Requests Completed
48
Total Tokens (Gen / Prompt)
Generated
3575
Prompt
42170
Avg Tokens per Request
74.5
Blocked Security Violations
33
User Perceived Latency (P95)
E2E Latency
TTFT
System Throughput (Tokens/s)
Throughput
Workload Distribution (Running vs Waiting)
Running
Waiting (Queued)
KV Cache Usage (Hybrid)
4.30%
vLLM CPU Usage
0.813%
Process Memory (RSS)
vLLM RAM (GB)
Cache Hit Rate (Prefix Caching)
71.9%